

MEE-FOLIO · TECHNICAL GUIDE

How your portfolio works

A section-by-section walkthrough of what each part of the site does, how it was built, the tools behind it, and which files to open when you want to change it.

Prepared for Kushagra · October 2026 · covers the code in E:\Mee-folio

Contents

Start here	3
Shared building blocks	5
Toolbar (navigation bar)	6
Hero: blueprint canvas and role cards	8
About: intro and halftone portrait	10
Work: projects on a design canvas	11
AI Work: Alignerr code evaluation	13
Experience: the metro map	14
Education & certifications: levels and achievements	15
Contact: email and resume download	16
The server and database	17
Archive, content checklist and next steps	18
Glossary	19

Start here

This guide explains every part of the site you see in the browser, from the toolbar at the top to the contact form at the bottom, plus the small server behind it.

Each chapter follows the same pattern: **what it does** (what a visitor sees), **how it works** (the logic, step by step), **built with** (the tools and browser features involved), **files** (where the code lives) and **how to change it** (the common edits you are likely to make).

The big picture

Mee-folio is one repository with two programs inside it. They are set up as npm workspaces, so a single `npm install` at the root installs both, and a single `npm run dev` starts both.

Part	Folder	What it is	Main tools
Client (the website)	<code>client/</code>	Everything the visitor sees. A single-page app: one HTML page that React fills in.	React 19, TypeScript 5.8, Vite 6, Motion 12
Server (the API)	<code>server/</code>	A small web API that stores projects and resume-form submissions in MongoDB.	Node.js, Express 5, Mongoose 8, MongoDB, dotenv, cors, tsx
Dev script	<code>scripts/dev.mjs</code>	Starts the client and server together and stops both when you press Ctrl+C.	Node child processes

How a page load works

- You run `npm run dev`. Vite serves the client on `http://localhost:5173`; Express serves the API on `http://localhost:5000`.
- The browser loads `client/index.html`, which loads `client/src/main.tsx`. That file mounts the `App` component into the page.
- `App.tsx` lays out every section in order: toolbar, Hero, About, Work, AI Work, Experience, Education & certifications, Contact, footer.
- Sections that need data from the server call `/api/...`. In development Vite *proxies* those calls to port 5000 (see `client/vite.config.ts`), so the browser never needs to know two servers exist.
- If the API is down, the site still works: Projects falls back to example projects, and the resume download still happens.

Tools used, in plain words

Tool	Role in this project
React	Builds the interface out of components (small reusable pieces like <code>RoleBoard</code> or <code>ContactSection</code>) and re-renders them when their state changes.
TypeScript	JavaScript with types. Catches mistakes (a misspelt field, a missing prop) before the code runs. <code>npm run typecheck</code> checks both client and server.
Vite	The development server and build tool. Instant reloads while you edit; <code>npm run build</code> produces the optimised files you deploy.
Motion (<code>motion/react</code>)	Animation library. Used for fade/slide reveals, layout transitions, the camera glide in Projects and springy details.
Canvas 2D	A browser drawing surface. Used where we draw thousands of things per frame: the hero blueprint, the halftone portrait, the archived particle hero.
Plain CSS	Each component has its own <code>.css</code> file next to it. No CSS framework; class names are prefixed per component (e.g. <code>canvas__frame</code> , <code>xs-m__dot</code>) so they don't clash.

Tool	Role in this project
Express	Minimal Node.js web server: receives HTTP requests and sends JSON back.
MongoDB + Mongoose	MongoDB stores documents (projects, leads). Mongoose defines their shape (a *schema*) and validates them.

Folder map

```

client/src/
  App.tsx           page layout, toolbar, section order
  main.tsx          mounts <App/>
  styles/global.css page-wide styles, toolbar styles
  content/profile.ts YOUR TEXT: intro, email, stats, experience, education...
  lib/canvasLoop.ts shared helper for canvas animations
  components/
    BlueprintHero/  hero canvas, lens and live CSS panel
    RoleBoard/      the "I am a ..." cycling cards
    About/          About text + halftone portrait
    ProjectShowcase/ Work section: canvas, viewer, navigator, data
    Evaluator/      AI Work section
    Experience/      Experience (metro) + Education (achievements)
    Contact/        email card, resume form, name check
  archive/RoleShapesHero/ alternative hero, kept for later (not in the build)
server/src/
  server.ts, app.ts start-up and route wiring
  models/           database shapes: Project, Lead
  controllers/      request handlers
  routes/           URL -> handler mapping
  scripts/          one-off commands (move a project to the end)

```

Most edits never touch component code. Your name, intro, email, Alignerr stats, roles, studies and certifications all live in `client/src/content/profile.ts`. Projects live in MongoDB.

Shared building blocks

A few pieces are used by many sections. Knowing them makes the rest of the guide easier to follow.

1. The content file: `content/profile.ts`

One TypeScript object holds all the personal text on the site: name, photo path, intro paragraphs, quick facts, email, resume file, Alignerr stats, roles, studies and certifications. Components import it and read from it, so changing your details means editing this one file. It ends with `as const`, which tells TypeScript to treat the values as fixed; that lets components get exact types (for example, the list of experience entries) for free.

2. Section reveal animation

In `App.tsx`, each section is a `motion.section` with a variant called `sectionReveal`: it starts slightly blurred, transparent and 48px lower, and animates to sharp, visible and in place when about 15–35% of it scrolls into view (`whileInView, viewport={{ once: true }}`). The `once` flag means it only plays the first time.

3. The canvas loop: `lib/canvasLoop.ts`

Canvas animations all need the same plumbing, so it lives in one function, `startCanvasLoop`. You give it a canvas, the section it belongs to, and an `onFrame` function. It then:

- **Sizes the canvas** to its section and for high-density screens (`devicePixelRatio`, capped at 2 for performance), and re-sizes it whenever the section changes size, using a `ResizeObserver`.
- **Tracks the pointer** in section coordinates (`x`, `y`, whether it is inside, whether a button is pressed, and its velocity).
- **Runs only while visible**. An `IntersectionObserver` starts `requestAnimationFrame` when the section is on screen and stops it when it isn't, so off-screen animations cost nothing.
- **Caps the time step** (`delta` is at most 1/30 s), so a background tab coming back doesn't make physics jump.
- Returns a cleanup function that removes every listener when the component unmounts.

4. Global styles: `styles/global.css`

Page-wide rules: base font and colours, the `site-shell` (content width max 1120px), the toolbar, section spacing (`.content-section`), the `eyebrow` label style and `scroll-margin-top`, so jumping to `#contact` lands below the toolbar instead of under it. The Work section uses `overflow-x: clip` to stop decorative glows from making the page scroll sideways.

5. Accessibility habits used throughout

- Interactive things are real `button` or `a` elements, so they work with the keyboard and screen readers.
- Visible focus rings (`:focus-visible`) on everything clickable.
- `aria-label`, `aria-pressed`, `aria-current` and `aria-live` describe state changes that are only visual otherwise.
- `prefers-reduced-motion`: visitors who ask their system for less motion get still or simplified versions of the big animations.

Toolbar (navigation bar)

What it does

A full-width bar styled like the top bar of a design tool. On the left: a small frame logo, **Mee-folio**, and a file-style breadcrumb such as / [about.tsx](#) that changes as you scroll. In the middle: tab-style links to About, Work and Contact; the tab for the section you are in is highlighted. On the right: an **Open to work** button that jumps to Contact. It slides away when you scroll down and comes back when you scroll up.

How it works

- **Hide on scroll:** a scroll listener in [App.tsx](#) compares the new scroll position to the last one. Scrolling down more than 8px hides the bar; scrolling up more than 8px (or being near the top) shows it. The check runs at most once per frame ([requestAnimationFrame](#) throttling). Hiding is just a CSS class that moves the bar up by its own height.
- **Which section am I in:** the [useActiveSection](#) hook watches every section with an [IntersectionObserver](#) and picks whichever one currently has the most of itself on screen. That id drives both the highlighted tab and the breadcrumb file name (a small map from id to name like `work → projects.tsx`).
- **Sticky:** the bar uses `position: sticky; top: 0`, so it stays at the top while the page scrolls under it, with a translucent blurred background ([backdrop-filter](#)).
- On phones the breadcrumb and Open to work button are hidden so the three links fit.

Built with

React state and effects, [IntersectionObserver](#), CSS sticky positioning and transforms.

Files

[client/src/App.tsx](#) (markup, [navItems](#), [sectionFiles](#), [useActiveSection](#), scroll logic) and [client/src/styles/global.css](#) (all toolbar styles).

Smooth scrolling and the scroll hint

- **Smooth wheel** ([lib/smoothScroll.ts](#), turned on in [App.tsx](#)): each mouse-wheel or trackpad event moves a target position, and every frame the page eases part of the way there, so scrolling glides instead of jumping. Inner scroll areas, ctrl+wheel zoom, the open image viewer and reduced-motion visitors keep normal scrolling. If something else moves the page (scrollbar drag, keyboard, a nav link) the glide stops and lets it win. Nav links still use CSS `scroll-behavior: smooth`.
- **Scroll hint** ([components/ScrollHint/](#)): if a visitor stays at the top for 7 seconds, a "Scroll down for more" pill with a bobbing arrow appears in the bottom-right corner (clicking it goes to About), and the page dips 120px and glides back once to show there is more below. Any wheel, touch, key or click stops the dip. The pill hides when they scroll, and after they scroll past 300px it never comes back.
- The toolbar ignores the dip ([isProgrammaticScroll\(\)](#)), so it doesn't hide and reappear.
- Timings are constants at the top of [ScrollHint.tsx](#): `WAIT_MS`, `NUDGE_PX`, `DONE_AFTER`.

How to change it

- Add a nav link: add an entry to [navItems](#) in [App.tsx](#) with the section's id.

- Change breadcrumb names: edit `sectionFiles` in `App.tsx`.
- Experience and Education are tracked for the breadcrumb but deliberately have no nav links.

SECTION 1

Hero: blueprint canvas and role cards

What it does

The first screen looks like a design tool's canvas: pixel rulers along the top and left, a faint 12-column layout grid, dashed guide lines, a size/device/fps badge, and a CSS panel showing the live styles of the front card. In the middle, an "I am a" box sits beside a stack of role cards that cycle every 5 seconds. Your cursor is a blue **lens**: inside it the page turns into a blueprint showing element outlines, padding, sizes, measurements and hidden notes.

How the role cards work ([RoleBoard](#))

- Five roles are defined at the top of [RoleBoard.tsx](#) (label + colour). They are absolutely positioned cards, stacked with small offsets and rotations so they look like a real deck.
- Every 5 seconds the front card plays a two-step animation: **lift** (moves up and tilts, 720ms) then **drop** (goes to the back, 760ms). The stack order array is rotated by one and each card's `z-index` is recomputed.
- In the hero the board runs with `interactive={false}`, so cards can't be dragged. The component still supports dragging and arrow-key moving when `interactive` is true (with a 5-second idle reset).
- It reports events through optional props: `onFrontRoleChange`, `onCardDragStart` and `onCardLand`. The hero doesn't need them; the archived particle hero uses `onFrontRoleChange`.

How the blueprint canvas works ([BlueprintHero](#))

Two transparent canvases sit over the hero, both driven by `startCanvasLoop`:

- **Background canvas** (behind the cards): each frame it draws the 12-column grid (same 1120px width and 24px gutters as the real layout), a dot grid, and dashed **guides** extended from the front card and the "I am a" box. Guides are read live from `getBoundingClientRect()`, so they follow the cards as they move.
- **Overlay canvas** (above everything): draws the rulers with tick marks every 10px and labels every 100px, plus markers for the cursor position. Then the **lens**.

The lens, step by step

- Each frame it finds the real elements on the page ("I am a" box, the top two cards, the CSS panel, the badge) and reads their position, size and padding with `getComputedStyle`.
- It clips drawing to a circle around the lens (`context.clip()`), fills it with blueprint blue and a grid, then draws `drawBlueprintScene`: numbered layout columns, rule-of-thirds and diagonal guides, slowly rotating measurement circles around the front card, hatching inside elements, dimension arrows for content width and viewport height, a title block ("MEE-FOLIO · HERO, sheet 01/04, drawn by Kushagra") and six hidden notes.
- On top of that it draws each element's box: padding areas in green, content in blue, a dashed outline, and a yellow tag such as `button.role-card 563x131`. The front card is drawn fully, the card behind it at 50%, the rest are hidden. A red measurement shows the gap between "I am a" and the front card.
- **Movement rules**: the lens follows the cursor smoothly (exponential easing). When the cursor leaves, it shrinks away where it is. and stays hidden until the cursor is back over the hero (so on touch screens the lens doesn't appear). When the cursor returns while the lens is hidden, it appears directly under the cursor instead of sliding across.
- On phones the lens radius is limited to 30% of the screen width.

The live CSS panel and badge

`useTopCardStyles` reads the front card's computed styles every 150ms. It is a small 260px panel showing only the values that change as cards cycle: border colour, shadow, rotation, z-index and position. Values that changed in the last 0.9s flash yellow, which is why rotation and colour light up when cards cycle. The badge's fps number counts frames over half-second windows.

Built with

React, Canvas 2D (paths, clipping, dashed lines, text), `getBoundingClientRect` and `getComputedStyle` for live measurements, `ResizeObserver/IntersectionObserver` via `canvasLoop`, CSS for the panel and badge.

Files

`components/BlueprintHero/BlueprintHero.tsx + .css`, `components/RoleBoard/RoleBoard.tsx + .css`, `lib/canvasLoop.ts`.

How to change it

- Roles: edit the `roles` array in `RoleBoard.tsx` (keep the colour `tone` one of blue, orange, purple, teal, yellow).
- Cycle speed: the `5_000` timer in the cycle effect of `RoleBoard.tsx`.
- Hidden notes in the lens: the `NOTES` array in `BlueprintHero.tsx` (`u/v` are positions as fractions of the hero's width and height).
- Lens size: `LENS_RADIUS` (190).

About: intro and halftone portrait

What it does

Left: your heading, two intro paragraphs and four quick-fact cards. Right: your photo printed as a **halftone** (a picture made of dots of different sizes, like newspaper printing) inside a "press proof" frame with crop marks. The dots fly in when you scroll to it, scatter away from the cursor, a small lens shows the real photo under the cursor, and clicking "develops" the full photo outward from where you clicked. When nobody is interacting, the print keeps slowly changing on its own.

How the halftone is made

- The photo is loaded into a hidden, tiny canvas: one pixel for every 7px dot cell (`SPACING = 7`), using "cover" cropping biased upward so faces stay in frame.
- `getImageData` reads each pixel's brightness. Brightness is stretched to the full 0–1 range, then turned into a dot radius: darker pixel, bigger dot. Very small dots are skipped.
- Each dot remembers its home position. The dots are filled with one diagonal gradient (blue → purple → pink), drawn as a single path, which is far faster than filling dots one by one.

How the motion works

- **Fly-in:** on the first build, every dot starts at a random far-away point and is pulled home by a spring, so the portrait assembles itself.
- **Spring physics:** each frame, every dot accelerates toward its home (strength varies per dot), loses some speed to damping, and moves. A tiny wobble keeps it alive.
- **Cursor:** dots within 70px of the cursor are pushed away (`REPEL_RADIUS`, `REPEL_FORCE`), and an 88px lens draws the real photo clipped to a circle.
- **Develop:** clicking expands a second circular clip from the click point until the whole photo shows. Clicking again shrinks it and kicks the dots outward.
- **Idle evolution:** about 1.2s after the cursor leaves, an `idle` value eases from 0 to 1. While idle, waves of heavier and lighter ink roll diagonally (dot radius × a moving sine wave), dots drift on a slow current, the gradient angle swings, and every 4.5–7.5s a soft ripple runs out from a random point. All of this fades out as soon as you hover again.
- Reduced motion: the portrait stays still; clicking reveals the photo instantly.

Built with

Canvas 2D (`getImageData`, clipping, gradients), a hand-written spring simulation, `ResizeObserver` and `IntersectionObserver`. The fact cards use Motion staggered reveals.

Files

`components/About/AboutSection.tsx` + `.css`, `components/About/HalftonePortrait.tsx`. Text comes from `profile.intro` and `profile.facts`.

How to change it

- Your photo: put it in `client/public/images/` and set `photoUrl` in `profile.ts`. A plain, light background gives the best halftone.
- Dot size: `SPACING` in `HalftonePortrait.tsx` (smaller = more, finer dots, more work per frame).

Work: projects on a design canvas

What it does

Each project's screenshots sit as frames on a dotted design canvas. A narrow panel on the left lists the images as numbers with thumbnails (numbered buttons along the bottom on phones). Picking one makes the view glide over to that frame, which gets corner handles and a 1440 × 900 size tag. Clicking the selected frame opens it full screen. Small **GitHub** and **Live demo** buttons are fixed in the canvas's bottom-right corner. Under the canvas are the tech-stack chips; hovering one shows how that technology was used in the project. On the right: title, description, USP, and a navigator < ■ ■ ■ > to move between projects.

Where the data comes from (`projectsData.ts`)

- `useProjects()` starts with six example projects (`fallbackProjects`) so the section never looks empty, then requests `/api/projects`.
- If the server returns projects, each is converted by `mapApiProject` into the shape the UI uses: images (featured image + up to 5 more), stack, `stackUsage`, links, and an accent colour picked from a palette by position.
- If the request fails or returns nothing, the examples stay. A warning is logged to the browser console, nothing breaks.
- Projects arrive sorted by the server: by `order` (lowest first), then newest first.

How the canvas camera works

- Frames are laid out in a grid, 3 per row (`FRAME_W` 360, `FRAME_H` 225, gaps 80/90). This is the "world".
- The whole world is one `motion.div` that is moved and scaled like a camera. For the selected frame, the code works out a scale that fits the frame in the visible area (minus the panel, the 58px toolbar strip and the 62px bottom bar), multiplied by the zoom step (`ZOOM_STEPS` 60%–150%), and the x/y that centre it.
- Motion animates `x`, `y` and `scale` together over 0.75s. When moving between frames (not when zooming) the scale uses keyframes [`current`, 78%, `target`], which gives the zoom-out-and-back-in feel of panning in Figma.
- Switching project fades the old world out and the new one in (`AnimatePresence` keyed by project id).
- The zoom percentage is the frame's on-screen width as a share of 1440px.

Full-size viewer (`ImageLightbox`)

- Rendered with `createPortal` into `<body>`. That matters: the section's reveal animation uses a CSS filter, which would otherwise trap a `position: fixed` overlay inside the section.
- Opens with a fade and scale. ← → or the arrow buttons change image, Esc or clicking the dark background closes it, page scrolling is locked while open, and keyboard focus returns to where it was afterwards.
- Clicking the image zooms to 2.2×; moving the mouse moves the zoom origin, which pans the zoomed image.
- Projects without real images show a branded "screenshot coming soon" placeholder.

Mock screenshots (`FakeScreen`)

Until real screenshots exist, each image shows a small mock interface drawn with HTML and CSS in the project's colour: six layouts (dashboard with bars, list, line chart, card grid, form, kanban board), chosen by image position. As soon as an image has a real URL, [FakeScreen](#) shows the real image instead.

Stack hover: "How X was used"

- Each stack chip is a button. Hover, keyboard focus or tap sets [hoveredTech](#); leaving clears it.
- [StackUsage](#) shows a dashed card with `project.stackUsage[technology]`, or "Details coming soon" if none was written. Motion animates it in and out.
- On wide screens the card appears below the USP; on narrow screens (where text comes first) it appears right under the chips instead, so it stays next to what you tapped. Both are rendered; CSS shows the right one.

Project navigator ([ProjectNav](#))

- Previous / next buttons wrap around. One segment per project; the current one is longer and in the project colour. Clicking a segment jumps straight there.
- While the pointer is over the section, the $\leftarrow \rightarrow$ keys switch projects (ignored while typing in a field or when the viewer is open), and a small key hint appears.
- It sits at the bottom-right of the right column. The column is given the canvas's height (`--canvas-height`) and the navigator uses `margin-top: auto`, so it always lines up with the canvas's bottom edge.

Built with

React, Motion (camera animation, presence transitions), [ResizeObserver](#) (canvas size), [createPortal](#), [fetch](#), CSS custom properties (`--accent`, `--canvas-height`).

Files

`components/ProjectShowcase/`: [ProjectShowcase.tsx](#) + [.css](#) (section and canvas), [projectsData.ts](#) (data + loading), [FakeScreen.tsx](#) + [screens.css](#), [ImageLightbox.tsx](#) + [.css](#), [ProjectNav.tsx](#).

How to change it

- Add or edit projects in MongoDB (see the Server chapter for the fields). Real image URLs replace the mock screens automatically.
- Order: projects with a lower `order` show first. To send one to the end: `npm run projects:move-last --workspace server -- "Hayday"`.
- Edit the example projects: `fallbackProjects` in [projectsData.ts](#).

SECTION 4

AI Work: Alignerr code evaluation

What it does

Left: a short explanation of your evaluation work, three counting-up stats (200+ tasks, 600+ hours, 8 months) and skill tags. Right: a dark "review console" that plays through sample reviews: code appears line by line, problem lines get highlighted, review notes appear one at a time, then a score fills in. Tabs let a visitor pick a task. A footnote says the tasks are illustrative because real client work is under NDA. Earnings are intentionally not shown.

How it works

- **Count-up stats:** `CountUp` waits until the number is 80% in view (`useInView`), then uses Motion's `animate(0, value)` over 1.6s with an ease-out curve, updating the text on every step.
- **Review console:** three sample reviews live in the `reviews` array (prompt, code lines, annotations with a line number + verdict pass/warn/fail, score, summary). A `stage` counter advances every 1.15s: each step reveals the next annotation and highlights its line; the final step reveals the score meter.
- When a review finishes, autoplay waits 3.2s and moves to the next one. Clicking a tab stops autoplay so the visitor can read.
- The console only animates while on screen, and reduced-motion visitors see each review complete immediately.

Files

`components/Evaluator/EvaluatorSection.tsx` + `.css`. Stats and skills come from `profile.evaluator`.

How to change it

Stats and skill tags: `profile.evaluator` in `profile.ts`. Sample reviews: the `reviews` array at the top of `EvaluatorSection.tsx`.

SECTION 5

Experience: the metro map

What it does

Your roles are stations on a "Career line", oldest on the left. The line changes colour from one role to the next. Your current role pulses with a red **You are here** tag, and a dotted line continues to **Next stop: Your team?**, which links to Contact. Clicking a station shows that role on a dark departures-style board: numbered badge, role, company, type, location, dates, duration, highlights and tools. On phones the line turns vertical.

How it works

- Roles come from `profile.experience` (newest first in the file) and are reversed so the line reads left to right through time. The current role is selected by default.
- The track between stations is each station's `::before` pseudo-element: an 8px bar coloured with a gradient from this role's colour (`--line`) to the next role's (`--next`). The last segment into the future is a repeating dashed gradient.
- The "You are here" pulse is a CSS keyframe animation on the dot's `box-shadow` that grows and fades.
- Durations such as "1 yr 9 mos" are calculated from the start and end months by `duration()` in `shared.tsx`. `end: null` means "Present" and counts up to today.
- The details board swaps with a short fade (`AnimatePresence` keyed by the selected station).
- Phones: a media query turns the station grid into one column and rotates the track bars to run vertically.

Files

`components/Experience/ExperienceSection.tsx`, `shared.tsx` (date helpers, heading), `Experience.css` (styles for both Experience and Education).

How to change it

Edit `experience` in `profile.ts`: `role`, `company`, `type`, `location`, `start` ("YYYY-MM"), `end` ("YYYY-MM" or null for current), `highlights` and `stack`. Put the newest role first.

SECTION 6

Education & certifications: levels and achievements

What it does

Studies are shown as game **levels**: LVL 1 (Class XII), LVL 2 (B.Tech), each with a progress bar that fills to "✓ Completed" and your grade, then a dashed LVL 3 "Always learning" whose bar keeps loading. Certifications are dark **Achievement unlocked** cards with metallic hexagon badges, newest first. They pop in one after another, a shine sweeps across on hover, and clicking opens the verification link. A greyed "Locked: Next certification" card closes the list.

How it works

- Levels: `profile.education` is reversed so the oldest is LVL 1. Each bar is a `motion.span` that animates its width from 0% to 100% the first time it scrolls into view, staggered by level.
- The "loading" bar is a CSS animation that eases between 22% and 62% width forever.
- Achievements: sorted newest first by date. A `motion.div` with stagger variants makes the cards pop in with a slightly bouncy scale.
- Hexagon badges use CSS `clip-path: polygon(...)`; the outer hexagon is a metallic gradient of the card's colour and the inner hexagon shows the issuer's initials.
- The hover shine is a skewed, semi-transparent white gradient in `::after` that slides across the card when hovered.

Files

`components/Experience/EducationSection.tsx`, `shared.tsx`, `Experience.css`.

How to change it

Edit `education` (degree, school, start, end, grade, notes) and `certifications` (name, issuer, date "YYYY-MM", credentialId, url) in `profile.ts`. Set each certification's `url` to its real verification page.

Contact: email and resume download

What it does

Left: heading, a line about what you're open to, and a status pill with your current local time in India. Right: two cards. **Send an email** shows your address with a Copy button and buttons that open a new message in Gmail, Outlook, Yahoo or the visitor's mail app. **Download my resume** asks for name and email, then downloads the PDF.

Email card

- Each provider has a "compose" web address with your email and a subject already filled in (for example Gmail's mail.google.com/mail/?view=cm&to=...&su=...). "Mail app" uses a [mailto:](#) link.
- A website can't see which email service a visitor uses, so it guesses: if they typed a Gmail address into the resume form, Gmail gets a blue outline; otherwise the button they used last time (remembered in [localStorage](#)) is highlighted.
- Copy uses the Clipboard API and shows "Copied ✓" for 1.8s; if copying isn't allowed it falls back to opening a [mailto:](#) link.
- The local time updates every 30 seconds via [Intl.DateTimeFormat](#) with the [Asia/Kolkata](#) time zone.

Resume form

- **Checks, in order:** name not empty, email matches a simple pattern, then the **name check** ([nameCheck.ts](#)): placeholders (test, asdf, john doe...), digits, no letters, the same character three times, keyboard runs (qwer, asdf) or long words without vowels.
- **Gentle note:** a suspicious name shows "Hmm, ... doesn't look like a real name. Is it yours?" with **Yes, that's me** (continues straight away) and **Let me fix it** (selects the name). It also appears when leaving the name field.
- **LED sweep:** each failed check replays a red light that runs along that field's border from left to right in 1.8s and leaves nothing behind. It is a [span](#) overlaid on the input whose background is a bright gradient band, masked so only a 2px ring shows (CSS [mask](#) with [exclude](#)), with [background-position](#) animated. A counter in the [key](#) makes React remount it, so the animation replays every time.
- **Sending:** the form POSTs name, email and a hidden **honeypot** field to [/api/leads](#). Bots tend to fill every field, so a filled honeypot is quietly ignored by the server.
- **Never blocks the download:** if the server is down or errors, the warning goes to the console and the resume still downloads. Only a 400 response (invalid data) stops it and shows the message.
- **Remembering visitors:** after downloading, the visitor's name is saved in [localStorage](#). Next visit the card says "Welcome back, {name}" with a direct download button and a "Not {name}?" link.

Files

[components/Contact/ContactSection.tsx](#) + [.css](#), [components/Contact/nameCheck.ts](#). Email, subject and resume path come from [profile.contact](#); the resume file is [client/public/resume.pdf](#).

The server and database

How it starts

`server.ts` connects to MongoDB first (`config/db.ts`) and only then starts listening on the port from `.env` (default 5000). If `MONGO_URI` / `MONGODB_URI` is missing or the database can't be reached, it exits with an error. `app.ts` turns on CORS for the client's origin, JSON parsing (up to 8 MB) and mounts the routes.

Routes

Method + path	What it does
GET <code>/api/health</code>	Reports <code>ok</code> and whether the database is connected.
GET <code>/api/projects</code>	All projects, sorted by <code>order</code> (lowest first), then newest first.
GET <code>/api/projects/:id</code>	One project.
POST <code>/api/projects</code>	Create a project. Requires title and description; at most 6 images including the featured one.
PUT <code>/api/projects/:id</code>	Replace a project's fields (same checks).
DELETE <code>/api/projects/:id</code>	Delete a project.
POST <code>/api/leads</code>	Save a resume-form visitor (name, email, source). Validates lengths and email, ignores honeypot hits, and allows at most 5 requests per IP per 10 minutes.

Important before deploying: the create/update/delete project routes have no login. Anyone who finds the API could change your projects. Add authentication (or remove those routes in production) before putting the site online.

Database models

Model	Fields
Project (<code>projects</code>)	<code>title*</code> , <code>description*</code> , <code>techStack</code> [text], <code>stackUsage</code> {technology: text}, <code>githubUrl</code> , <code>liveUrl</code> , <code>featuredImageUrl</code> , <code>imageUrls</code> (max 5), <code>imageUrl</code> (older field, still read), <code>featured</code> (true/false), <code>order</code> (number, default 0), plus automatic <code>createdAt/updatedAt</code> . *required
Lead (<code>leads</code>)	<code>name</code> , <code>email</code> (stored lower-case), <code>message</code> (optional), <code>source</code> (<code>resume</code> or <code>contact</code>), plus timestamps.

Project order and the move-last command

Every project has an `order` number (0 by default). Lower numbers show first; projects with the same number show newest first. To send a project to the end, run this from the repo root on your computer:

```
npm run projects:move-last --workspace server -- "Hayday"
```

The script (`server/src/scripts/move-project-last.ts`) finds the one project whose title contains that text (case-insensitive), sets its `order` to one more than the current highest, and prints what it did. If no project, or more than one, matches, it says so and changes nothing. You can also set `order` by hand in MongoDB Compass.

Archive, content checklist and next steps

Archived hero: [RoleShapesHero](#)

An alternative hero kept for later in [client/src/archive/](#). Nothing imports it, so it isn't in the build. About 7,000 dots (2,600 on phones) form a picture of whichever role card is in front: a code review with a red cross and green tick, a page wireframe, client → API → database with moving "requests", a turning spirograph, and a growth chart.

- Each picture is drawn once onto a hidden half-size canvas in marker colours (black, red, green); every coloured pixel becomes a candidate dot position, and the colour decides the dot's tone.
- On a role change each dot waits a delay based on its new x position, then springs to its target, so the old picture dissolves into the new one left to right.
- To try it: in [App.tsx](#) replace `<BlueprintHero />` with `<RoleShapesHero />` (see [client/src/archive/README.md](#)). It needs portrait versions of the pictures before use on phones.

Content checklist

What	Where
Photo	Image in client/public/images/ , path in profile.photoUrl
Intro text, quick facts	profile.intro , profile.facts
Resume	Replace client/public/resume.pdf
Alignerr stats and skills	profile.evaluator
Roles	profile.experience
Studies, certifications (+ real verify links)	profile.education , profile.certifications
Projects, screenshots, stack usage text	MongoDB projects collection

Known gaps and suggested next steps

- Add login to the project-editing API before deploying (see the Server chapter).
- Add a proper [usp](#) field to projects; today the USP line for database projects is generated ("Built with ...").
- The server exits if MongoDB isn't configured, but the README says MongoDB is optional; decide which is right.
- Untrack the [.chrome-tab-return-check/](#) folder (a Chrome profile) from git and add it to [.gitignore](#).
- The current work isn't committed to git yet.

Useful commands

```

npm install          install everything (run once, at the repo root)
npm run dev          start client (5173) + server (5000)
npm run typecheck    check types in client and server
npm run build        production build of both
npm run projects:move-last --workspace server -- "Title"  send a project to the end

```

Glossary

Short definitions of the technical terms used in this guide.

Term	What it means here
Component	A reusable piece of interface written as a function that returns markup, e.g. <code>ContactSection</code> .
State	Data a component remembers between renders (e.g. which image is selected). Changing it re-renders the component.
Props	Inputs passed into a component, e.g. <code>interactive={false}</code> on <code>RoleBoard</code> .
Hook	A React function starting with <code>use...</code> that adds behaviour, e.g. <code>useState</code> , <code>useEffect</code> , or our own <code>useProjects</code> .
Canvas 2D	A browser drawing surface you paint on with code (shapes, text, images) every frame. Fast for thousands of dots.
<code>requestAnimationFrame</code>	Asks the browser to run a function right before the next screen refresh (about 60 times a second). Used for smooth animation.
<code>IntersectionObserver</code>	Browser feature that reports when an element enters or leaves the screen. Used to start/stop animations and track the current section.
<code>ResizeObserver</code>	Browser feature that reports when an element changes size. Used to resize canvases.
<code>getBoundingClientRect</code> / <code>getComputedStyle</code>	Read an element's on-screen position and size, and its final CSS values. The hero's lens and CSS panel use these to show real measurements.
Spring physics	Each dot is pulled toward a target with a force proportional to the distance and slowed by damping. Gives natural, bouncy motion.
Clip / clipping	Restricting drawing to a shape, e.g. only inside the lens circle.
Portal	Rendering a component somewhere else in the page (here: directly in <code><body></code>) while keeping it in the same React tree.
Proxy (Vite)	In development, Vite forwards <code>/api</code> requests from port 5173 to the server on 5000.
Honeypot	A form field hidden from people. Bots fill it in; the server ignores submissions where it isn't empty.
Rate limit	A cap on how often one visitor can call an endpoint (5 resume submissions per 10 minutes per IP).
<code>localStorage</code>	Small storage in the visitor's browser. Used to remember the resume visitor's name and their last mail provider.
CSS custom property	A CSS variable such as <code>--accent</code> , set per project or per role so colours follow the content.
Media query	CSS that applies only at certain screen sizes, e.g. turning the metro line vertical on phones.
<code>prefers-reduced-motion</code>	A visitor's system setting asking for less animation; the site respects it.